

YangtzeDelta Proposal Evaluation Technical Guide

A comprehensive reference for the four evaluation strategies used to assess AI-generated proposal reports — covering methodology, metrics, formulas, inputs, and outputs.

PROJECT

YangtzeDeltaApplication

PIPELINE STAGES

9-Stage LLM Pipeline

CORE STRATEGIES

3 Core + 1 Supporting

GENERATED

June 2026



Executive Overview

Pipeline architecture, evaluation goals, and the relationship between the four strategies.

The YangtzeDelta pipeline is a 9-stage AI system that evaluates business, startup, and research proposals across five dimensions — **team, objectives, strategy, advantages, and feasibility**. Each proposal passes through fact extraction, question generation, LLM answering, expert opinion synthesis, and final report generation. To measure the quality of these AI-generated evaluations, four complementary strategies are applied: an agreement test against human expert scores, a hallucination detection pass, a logical coherence test under controlled proposal changes, and a stochastic stability sweep. Together they form a multi-dimensional quality assurance framework.

STRATEGY 1 · CORE

Cohen's Kappa Agreement

```
evaluate_cohens_kappa.py
```

Measures inter-rater reliability between human expert scores and AI-generated scores across all evaluation dimensions.

STRATEGY 2 · CORE

Groundedness Evaluation

```
evaluate_groundedness.py
```

Detects hallucinations by verifying that every factual claim in the AI report is traceable back to the original proposal text.

STRATEGY 3 · CORE

Sensitivity Analysis

```
evaluate_sensitivity.py
```

STRATEGY 4 · SUPPORTING

Sweep Consistency

```
evaluate_sweep.py
```

Tests logical coherence — whether AI scores respond correctly in direction, magnitude, and dimension when the proposal is deliberately varied.

Quantifies stochastic variance by running the pipeline (or score extraction) multiple times on the same proposal and measuring score stability.

Pipeline Stage Map

STAGE	SCRIPT	OUTPUT
Stage 0	<code>prepare_proposal_text.py</code>	Cleaned full text
Stage 1	<code>extract_facts_by_chunk.py</code>	Fact chunks per dimension
Stage 2	<code>build_dimensions_from_facts.py</code>	<code>dimensions_v2.json</code>
Stage 3	<code>generate_questions.py</code>	Question set per dimension
Stage 4	<code>llm_answering.py</code>	Structured answers (CoT/ReAct/SC)
Stage 5	<code>post_processing.py</code>	Refined answer payload
Stage 6	<code>ai_expert_opinion.py</code>	Expert opinion JSON
Stage 7	<code>generate_final_report.py</code>	<code><pid>_final_report.md</code>
Stage 8	<code>generate_llm_scores.py</code>	<code>llm_scores.json</code>

Unified Score Scale

All four evaluation strategies operate on a shared **0–1 normalized scale** to enable cross-strategy comparison. The mapping rules are:

Human Ordinal Scores (1–5)

Applies to dimensions: team, objective, strategy, advantages, feasibility

$$\text{normalized} = (\text{score} - 1) / 4$$

Ranking Scores (1–20)

Applies to the overall_ranking dimension only

$$\text{normalized} = (\text{score} - 1) / 19$$

AI float scores extracted from `final_report.md` are already on the 0–1 scale and used directly without transformation.



Strategy 1 — Cohen's Kappa Agreement

Measures statistical agreement between AI-generated scores and human expert scores to quantify inter-rater reliability.

i Goal: Determine whether the AI evaluation system produces scores that a human expert would endorse. A high kappa means the AI can reliably substitute for (or augment) a human reviewer.

Inputs & Outputs

TYPE	PATH
Input	src/data/evaluations/human/human_scores.xlsx
Input (primary)	src/data/llm_scores/<pid>/llm_scores.json
Input (fallback)	src/data/reports/<pid>_final_report.md

TYPE	PATH
Output	src/data/evaluations/runs/<run_id>/evaluation_summary.json
Output	src/data/evaluations/runs/<run_id>/evaluation_report.xlsx

Dimensions Evaluated

Six scoring dimensions plus one binary verdict are evaluated:

DIMENSION	HUMAN SCALE	NORMALIZED RANGE	PRIMARY METRIC
team	1 – 5 ordinal	0.00 – 1.00	Weighted Cohen's K
objective	1 – 5 ordinal	0.00 – 1.00	Weighted Cohen's K
strategy	1 – 5 ordinal	0.00 – 1.00	Weighted Cohen's K
advantages	1 – 5 ordinal	0.00 – 1.00	Weighted Cohen's K
feasibility	1 – 5 ordinal	0.00 – 1.00	Weighted Cohen's K
overall_ranking	1 – 20 ranking	0.00 – 1.00	ICC(2,1) + Spearman

DIMENSION	HUMAN SCALE	NORMALIZED RANGE	PRIMARY METRIC
verdict	Y / N binary	categorical	Exact match accuracy

Binning for Kappa Computation

Continuous 0–1 scores are discretized into 5 equal bins before computing kappa, mapping to the label set `{0.0, 0.25, 0.5, 0.75, 1.0}` :

5-BIN DISCRETIZATION

```
bin = round(score × 4) / 4
```

→ maps $[0.0-0.125) \rightarrow 0.0$, $[0.125-0.375) \rightarrow 0.25$, ... $[0.875-1.0] \rightarrow 1.0$

Statistical Metrics

Weighted Cohen's κ

Primary agreement metric. Uses linear weights so adjacent-bin disagreements count less than extreme disagreements. Range: -1 to $+1$. $\kappa \geq 0.61$ = substantial agreement.

Unweighted Cohen's κ

Treats all disagreements equally. More conservative than weighted κ . Useful for detecting systematic misclassification patterns.

ICC(2,1) — Intraclass Correlation

Applied to `overall_ranking` only. Two-way random effects model for absolute agreement. Preferred over kappa for continuous rankings.

Spearman ρ + p-value

Non-parametric rank correlation. Captures monotonic agreement without assuming a linear relationship. Reported with two-tailed p-value.

95% Bootstrap CI

Confidence intervals for kappa and ICC computed via 1000-iteration bootstrap resampling. Quantifies estimation uncertainty for small sample sizes.

Bias Analysis

Mean difference (AI – Human) per dimension. Paired t-test p-value tests whether the mean difference is significantly non-zero (systematic over- or under-scoring).

Core Formula

WEIGHTED COHEN'S KAPPA

$$\kappa = (p_o - p_e) / (1 - p_e)$$

p_o = observed agreement proportion (weighted)

p_e = expected agreement by chance (weighted)

Bias: $mean_diff = mean(AI_scores) - mean(Human_scores)$

T-test: $H_0: mean_diff = 0 \rightarrow p < 0.05 = significant\ bias$

Interpreting Kappa Values

K RANGE	INTERPRETATION	BADGE
< 0.20	Slight agreement	Poor
0.21 – 0.40	Fair agreement	Fair
0.41 – 0.60	Moderate agreement	Moderate

K RANGE	INTERPRETATION	BADGE
0.61 – 0.80	Substantial agreement	Substantial
0.81 – 1.00	Almost perfect agreement	Excellent



Strategy 2 — Groundedness Evaluation

Verifies that AI-generated report claims are traceable to the source proposal text, measuring factual accuracy and hallucination rate.

i **Goal:** Detect hallucinations — claims made by the AI that are absent from, or contradict, the original proposal. A high groundedness score means the report faithfully reflects the proposal content.

Inputs & Outputs

TYPE	PATH
Input	src/data/prepared/<pid>/full_text.txt
Input	src/data/reports/<pid>_final_report.md
Output	src/data/evaluations/runs/<run_id>/groundedness_result.json

Two-Stage Process

1 Claim Extraction

LLM call at **temperature 0.0**, max_tokens 1800. The model reads Sections 0 & 1 of the final report and extracts a flat list of distinct factual claims (e.g., "The team has 15 years of combined experience in biotech"). Deterministic settings ensure reproducible extraction.

2 Claim Classification

Each extracted claim is classified against the proposal text into one of three categories. For supported and inferred claims, the exact proposal quote is stored as evidence. Classification LLM also runs at temperature 0.0 to minimize inconsistency.

Classification Categories

CATEGORY	DEFINITION	WEIGHT IN LENIENT SCORE
supported	Claim is explicitly stated in the proposal text — a direct quote or paraphrase can be found.	1.0
inferred	Claim is a reasonable implication from the proposal context but is not explicitly written. Represents reasonable interpretation rather than fabrication.	0.5
unsupported	Claim is absent from the proposal, contradicts it, or introduces information not found anywhere in the source text. Hallucination risk.	0.0

Score Formulas

THREE GROUNDEDNESS METRICS

```
groundedness_score = n_supported / n_total ← strict: only  
explicit support counts  
  
lenient_score = (n_supported + 0.5 × n_inferred) / n_total ←  
partial credit for reasonable inferences  
  
hallucination_rate = n_unsupported / n_total ← fraction of  
claims with no proposal basis
```

Output Structure

The `groundedness_result.json` contains a record for every extracted claim:

```
{  
  "claim": "The founding team holds three FDA 510(k) clearances.",  
  "classification": "supported",  
  "evidence_quote": "Our team secured three 510(k) clearances between  
  "groundedness_score": 0.82,  
  "lenient_score": 0.89,  
  "hallucination_rate": 0.11  
}
```

Text Truncation Rationale

Proposal text is capped at **24 000 characters** and report text at **20 000 characters** before LLM calls. This controls API cost and prevents context overflow on long proposals, while retaining the factual density sections most likely to contain verifiable claims.



Strategy 3 — Sensitivity Analysis

Tests whether AI scores respond logically and proportionally to deliberate, controlled changes in proposal content.

- i Goal:** Verify that the evaluation pipeline is not outputting scores arbitrarily — when a proposal is modified to strengthen or weaken a specific dimension, only the appropriate dimensions should change, in the correct direction and by a proportional magnitude.

Inputs & Outputs

TYPE	PATH
Input	src/data/reports/<baseline_pid>_final_report.md
Input	src/data/reports/<variant_pid>_final_report.md
Input	src/data/llm_scores/<pid>/llm_scores.json
Output	src/data/evaluations/runs/<run_id>/sensitivity_summary.json

TYPE	PATH
Output	src/data/evaluations/runs/<run_id>/sensitivity_report.xlsx

Analysis Process

- 1

Score Delta Computation

For each evaluation dimension, compute `delta = variant_score - baseline_score` on the 0–1 scale. Deltas with absolute value < 0.01 are treated as noise and filtered out (MIN_DELTA_THRESHOLD).
- 2

Report Excerpt Preparation

Sections 0 & 1 of both reports are extracted (capped at 16 000 chars each) and passed to the LLM for contextual comparison alongside the numeric deltas.
- 3

LLM Coherence Assessment

An LLM call at **temperature 0.1**, max_tokens 800 judges the pattern of changes. It is given the intended modification description, the score deltas, and report excerpts, then assesses whether the changes make logical sense.
- 4

Verdict Aggregation

Per-variant results are aggregated into an overall sensitivity rating using a weighted point scheme.

LLM Assessment Fields

FIELD	TYPE	VALUES	MEANING
sensitivity_quality	enum	good partial poor	Overall quality of the sensitivity response

FIELD	TYPE	VALUES	MEANING
<code>direction_assessment</code>	enum	correct mixed incorrect	Whether score changes went in the right direction
<code>appropriately_sensitive</code>	bool	true / false	Whether overall change magnitude was proportional
<code>correctly_affected</code>	list	dimension names	Dimensions that changed correctly
<code>unexpected_changes</code>	list	dimension names	Dimensions that changed when they should not have
<code>missed_changes</code>	list	dimension names	Dimensions that should have changed but did not

Scoring & Thresholds

SENSITIVITY POINT SCHEME & AGGREGATION

Per-variant points:

`sensitivity_quality == "good" → 2 pts`

`sensitivity_quality == "partial" → 1 pt`

`sensitivity_quality == "poor" → 0 pts`

Overall rating from average points:

`avg ≥ 1.7 → good`

avg $\geq$ 0.8 $\rightarrow$ partial

avg $<$ 0.8 $\rightarrow$ poor

Noise floor: $|\text{delta}| < 0.01 \rightarrow \text{ignored}$

Example Variant Types

MODIFICATION	EXPECTED DIMENSION CHANGE	EXPECTED DIRECTION
Increased TAM from \$50M to \$500M	strategy, advantages	$\uparrow$ upward
Removed PhD from team bio	team	$\downarrow$ downward
Added regulatory approval milestone	feasibility	$\uparrow$ upward
Removed financial projections section	strategy, feasibility	$\downarrow$ downward
Changed objective from research to commercial	objective	$\leftrightarrow$ context-dependent



Strategy 4 — Sweep Consistency

Quantifies stochastic variance by running the same proposal through the pipeline multiple times and measuring score stability.

i Goal: Determine how much the pipeline's outputs vary due to LLM non-determinism. Low variance (high consistency) means users can trust that a single run is representative. High variance flags that multiple runs should be averaged before making decisions.

Inputs & Outputs

TYPE	PATH	D
Input	src/data/prepared/<pid>/	P p p d u: fi al
Input	src/data/reports/<pid>_final_report.md	E re if se fl
Output	src/data/evaluations/runs/<run_id>/sweep_summary.json	A st ac ru
Output	src/data/evaluations/runs/<run_id>/sweep_report.xlsx	P ac

TYPE	PATH	D
		sc
Output	src/data/evaluations/runs/<run_id>/sweep_bundle.zip	A ru fo

Run Modes

Full Pipeline Mode (default) Runs Stages 1–7 (or 1–8 with <code>--use_llm</code>) in full for each sweep iteration. Captures end-to-end variance including fact extraction, answer generation, and report synthesis. Each run's <code>final_report.md</code> is archived to <code>run_N/</code> .
Scores-Only Mode (<code>--scores_only</code>) Re-extracts scores from the existing <code>final_report.md</code> without re-running the pipeline. Score extraction is deterministic (regex-based), so this mode always produces zero variance — used only to confirm the extraction mechanism is stable.

Metrics per Dimension

METRIC	FORMULA	INTERPRETATION
Mean	$\Sigma(\text{scores}) / N$	Average score across N runs — the best point estimate
Std Dev (σ)	$\sqrt{(\Sigma(x_i - \bar{x})^2 / N)}$	Absolute spread of scores — lower is more stable
CV (Coeff. of Variation)	σ / mean	Relative spread — enables comparison across dimensions with different means
Min / Max / Range	$\text{max} - \text{min}$	Worst-case spread across all runs

METRIC	FORMULA	INTERPRETATION
Mode	most frequent value	Most likely output for deterministic reporting
Exact match count	runs where score == mode	How often the pipeline reproduces the same result
All-exact-match	boolean	True only if every run produced an identical score

Stability Interpretation Guidelines

CV RANGE	STABILITY	RECOMMENDATION
0.00 – 0.05	Excellent	Single run is reliable for reporting
0.05 – 0.10	Good	Single run is acceptable; confirm with 2–3 runs for critical decisions
0.10 – 0.20	Moderate	Average 3–5 runs before reporting final scores
> 0.20	High Variance	Pipeline is unstable for this dimension; investigate prompt or temperature settings

CV FORMULA FOR SWEEP ANALYSIS

$$CV = \sigma / \text{mean}$$

Default runs: $N = 5$

Score scale: 0–1 float (extracted from `final_report.md`)

```
markdown table)
```

```
Verdict: mode + exact_match_rate (categorical, not numeric)
```



Supporting Infrastructure

Prompt strategies, LLM providers, score extraction, and multi-provider orchestration that underpin all four evaluation strategies.

Stage 4 Prompt Strategies

Three reasoning frameworks are available in `src/tools/prompt_templates.py`, selectable per pipeline run:

COT

Chain-of-Thought

- 7-step structured evaluation
- Understand → Team → Objectives → Strategy → Advantages → Feasibility → Synthesize
- Confidence range: 0.50 – 0.92
- Output: answer, claims, evidence_hints, general_insights, topic_tags

REACT

ReAct-Inspired

- Internal Thought → Action → Observation loop
- 5 attributes: Team, Objective, Strategy, Advantages, Feasibility
- Evidence: CONFIRMED / PARTIAL / PLANS / NOT VISIBLE
- Risk tiers: CRITICAL / IMPORTANT / MINOR

SC

Self-Consistency

- 5 internal perspectives checked silently
- Factual, Attribute, Risk, Stakeholder, Action
- Aggregated to single JSON answer

→ Handles nuanced / mixed results gracefully

LLM Providers

All pipeline stages and evaluation tools support three providers, configured via `.env` :

OpenAI gpt-4o-mini · JSON mode ✓ · Batch ✓

Gemini gemini-2.5-flash · JSON mode ✓ · Batch ✓

DeepSeek deepseek-chat · JSON mode ✕ · Single-question only

Score Extraction (`score_extraction.py`)

Scores are extracted from the final report's markdown table using regex patterns. This is the fallback input for all evaluation strategies when `llm_scores.json` is not available.

DIMENSION	REGEX PATTERN	DEFAULT IF MISSING
team	<code>(team)\s*\ \s*([\d.]+)</code>	0.5
objective	<code>(objectives)\s*\ \s*([\d.]+)</code>	0.5
strategy	<code>(strategy)\s*\ \s*([\d.]+)</code>	0.5
advantages	<code>(innovation)\s*\ \s*([\d.]+)</code>	0.5
feasibility	<code>(feasibility)\s*\ \s*([\d.]+)</code>	0.5
verdict	<code>GO → "Y", NO-GO / HOLD → "N"</code>	—

Multi-Provider Orchestration

`run_multi_provider_lifecycle.py` runs the full Stage 0–8 cycle independently for each configured provider in parallel using `ThreadPoolExecutor`. Each provider gets a unique proposal ID suffix:

```
<base_pid>__openai
<base_pid>__gemini
<base_pid>__deepseek
```

After all runs complete, the preferred provider (from `PROVIDER` env var) is promoted to the canonical compatibility path. A lifecycle summary is written to `provider_lifecycles/<pid>/provider_lifecycle_summary.json`.

Concurrency Configuration

ENV VARIABLE	DEFAULT	CONTROLS
<code>STAGE4_DIM_WORKERS</code>	2	Parallel dimensions per provider in Stage 4
<code>STAGE4_VARIANT_WORKERS</code>	3	Parallel variants (default/risk/implementation) per batch
<code>STAGE4_REFINE_WORKERS</code>	3	Parallel refinement passes
<code>STAGE4_PROVIDER_WORKERS</code>	3	Parallel providers in multi-provider mode

Answer Normalization Pipeline (Stage 4)

Before scores are extracted, Stage 4 answers go through a 5-step normalization chain in `llm_answering.py`:

1 Candidate Normalization

Converts list/dict answers to numbered bullet format. Calibrates confidence to 0.50–0.92 range.

2 Redline Detection

Moves concrete facts (dates, money amounts, trial IDs like NCT numbers, DOIs, patents, ISO standards) from `claims` → `evidence_hints` to prevent overly specific unverified assertions.

3 General Insights Scrubbing

Ensures `general_insights` contains only baseline/practice statements, moving specific facts to `evidence_hints`.

4 Bullet Formatting

Normalizes to "1. 2. 3." numbered format. Removes Markdown markers. Caps at 8 bullets per answer.

5 Deduplication

Uses simhash on answer text to remove near-duplicate answers across variants. Eliminates low-confidence duplicates first.



LLM-as-a-Judge Pattern

Two evaluation strategies use a second LLM to assess the quality and correctness of the primary pipeline's outputs — a pattern known as LLM-as-a-Judge.

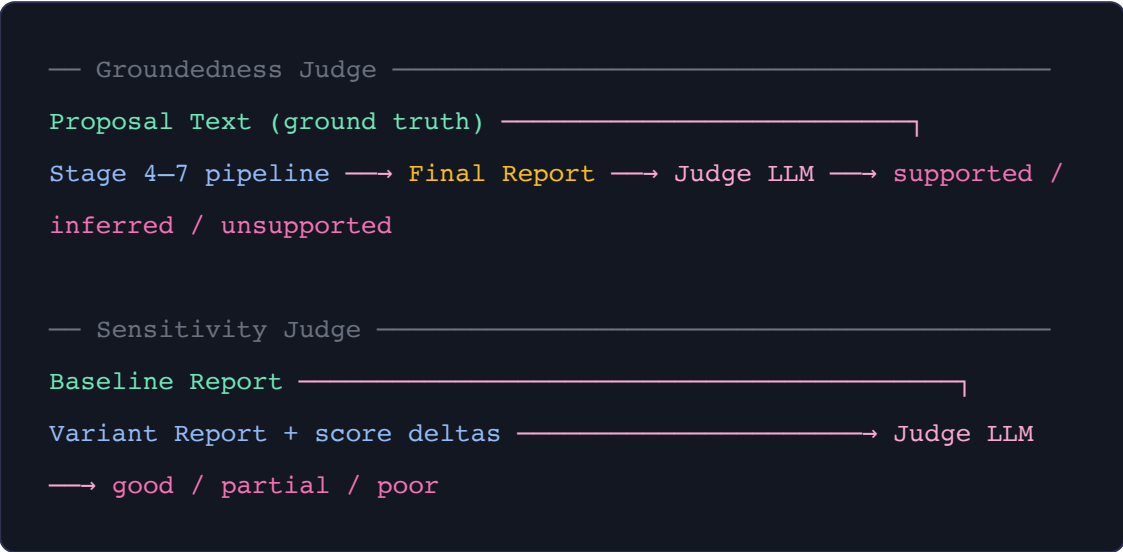
i What is LLM-as-a-Judge? Instead of relying only on statistical metrics or human labels, an LLM is given both the AI-generated output *and* a reference source, then asked to render a structured verdict on whether the output is correct, faithful, or coherent. The judge LLM never sees the intermediate reasoning — only the final artifact and the reference material it should be grounded in.

Where the Pattern Appears

STRATEGY	FILE	LLM-AS-JUDGE?	WHAT IS JUDGED
Groundedness	<code>evaluate_groundedness.py</code>	Yes	Are report claims traceable to the proposal?
Sensitivity Analysis	<code>evaluate_sensitivity.py</code>	Yes	Are score changes logically coherent?
LLM Scoring (Stage 8)	<code>generate_llm_scores.py</code>	No — LLM-as-Scorer	Applies a rubric directly to the proposal; does not

STRATEGY	FILE	LLM-AS-JUDGE?	WHAT IS JUDGED
			assess another LLM's output
Cohen's Kappa	<code>evaluate_cohens_kappa.py</code>	No — Statistics only	Pure math (κ , ICC, Spearman) — zero LLM calls
Sweep Consistency	<code>evaluate_sweep.py</code>	No — Variance analysis	Score stability across runs — no judgement

Architecture



Judge 1 — Groundedness: Exact Prompts

Two sequential LLM calls, both at **temperature 0.0** (fully deterministic) in

`evaluate_groundedness.py` :

CALL 1 — CLAIM EXTRACTION

SYSTEM PROMPT

"You are a claim extraction evaluator. Your task: extract factual claims that the PROPOSAL ITSELF explicitly makes, as reported in the evaluation summary."

USER PROMPT (ABBREVIATED)

"Extract factual claims that the PROPOSAL explicitly states, as summarised in the evaluation report below...
Return exactly this JSON: {"claims": [{"claim_id": 1, "claim": "string"}]}"

CALL 2 — CLAIM CLASSIFICATION

SYSTEM PROMPT

"You are a groundedness evaluator. Verify whether each claim is grounded in the provided business proposal text. Use ONLY the proposal text — no outside knowledge. Classify each claim as exactly one of: 'supported', 'inferred', or 'unsupported'..."

RETURNS PER CLAIM

```
{"claim_id", "claim", "classification":  
"supported|inferred|unsupported",  
"supporting_quote": "<exact quote from proposal>", "reason":  
"..."}"
```

Judge 2 — Sensitivity: Exact Prompt

Single LLM call at **temperature 0.1** (near-deterministic) in

`evaluate_sensitivity.py` :

SYSTEM PROMPT

"You are an expert evaluator assessing whether an AI proposal-analysis system is appropriately sensitive to changes in business proposal content and metrics. You will receive: 1. Score changes (variant minus baseline) across rubric dimensions, 2. A baseline report excerpt, 3. A variant report excerpt. Your job: judge whether the score shifts are logically coherent with the differences visible in the two reports – as a domain expert would expect."

USER PROMPT ASKS THE JUDGE TO ASSESS

1. Are score changes directionally correct?
2. Did the RIGHT dimensions change?
3. Did any dimension change for NO visible reason?
4. Did any dimension FAIL to change when it clearly should have?
5. Is magnitude proportional?

RETURNS

```
{"sensitivity_quality": "good|partial|poor",  
"direction_assessment": "correct|mixed|incorrect",  
"appropriately_sensitive": true|false,  
"dimensions_correctly_affected": [...],  
"unexpected_changes": [...],  
"missed_changes": [...],  
"key_observations": [...],  
"overall_verdict": "..."}  

```

Temperature Design Rationale

Groundedness: temp = 0.0

Claim classification is a factual lookup task — either the quote is in the proposal or it isn't. Temperature 0.0 gives a deterministic result, ensuring that running the evaluator twice on the same proposal yields identical claim verdicts. Reproducibility is essential for audit trails.

Sensitivity: temp = 0.1

Coherence judgment requires some interpretive flexibility — the judge must reason about *why* a dimension should or should not change, which benefits from slight variability to avoid rigid pattern-matching. A small non-zero temperature allows more natural reasoning while remaining nearly deterministic.

Key Design Principles

1 Separation of concerns

The judge LLM is a different call from the pipeline LLM. It receives only the artifact and reference — never the intermediate prompt chain that produced the artifact.

2 Structured outputs only

Both judge calls return strict JSON schemas. This enables downstream statistical aggregation (groundedness score, kappa inputs) rather than free-text summaries that require further parsing.

3 No outside knowledge

The groundedness judge is explicitly instructed: "Use ONLY the proposal text — no outside knowledge." This prevents the judge from hallucinating support for claims based on general domain knowledge rather than the actual proposal.

4 Provider-agnostic

Both judge calls use the same `llm_provider.py` abstraction as the pipeline. The judge can run on OpenAI, Gemini, or DeepSeek — configurable via `GROUNDEDNESS_MODEL` env var or provider default.